

16th Progress Report

W. Chan

Date:	17/06/2024
Subject:	RL for Flying-V
Supervisors:	Dr. E.J. van Kampen
Project term:	01/2024 - 08/2024
E-mail:	w.y.chan@student.tudelft.nl

Outcome of the previous meeting

- **Discussed two main options for future steps** of research and their utility and risks:
 - Option 1: Developing lateral controller, then using a longitudinal + lateral rate tracking task to arrive at answers for the research questions.
 - Option 2: Extend development of longitudinal controller, get an *attitude* tracking controller working, an even further extension would be getting *altitude* tracking controller working. Then using the attitude/altitude longitudinal controller to answer the research questions.

I think the more academically interesting option is **option 2**, it might be difficult for IDHP to achieve good tracking performance here, so to **a)** see if I can get IDHP to work for attitude/altitude tracking, and then to **b)** see if the augmentations can improve the algorithm's ability to perform attitude/altitude tracking. So I will proceed with this option.

- **Discussed the RLS estimator and the model which it estimates**, specifically concerning the utility and validity of estimating an incremental tracking error model.

From a modelling perspective, such a model goes along the lines of trying to find the relationship between subsequent tracking errors. This idea is a bit dubious if one thinks a little deeper, since it supposes that by knowing the tracking error at one time step one can predict what the tracking error of the next time step is. For instance, whether the error will be higher or lower at the next time step, which is an estimation that could be biased by observations: seeing the error go down over the previous time step might cause the model to predict an ever-decreasing error, but such an observation should not necessarily mean a decreased error at the next time step.

However, finding a model for the tracking error is useful and arguably logical from the IDHP or adaptive dynamic programming perspective. Since the use of the RLS estimated model is to find the relationship between:

- The MDP state variable and the control action i.e. the agent policy $\frac{\partial s}{\partial u}$.
- The MDP state variable between subsequent time steps $\frac{\partial s_{t+1}}{\partial s_t}$, or equivalently $\frac{\partial s_t}{\partial s_{t-1}}$.

Furthermore, estimating such a model can be practically implemented, one simply has to feed the increment of the MDP state to the RLS estimator. But more importantly, doing so could reduce the complexity of the learning task for the IDHP critic, as one can restrict the IDHP critic to solely estimate the value function gradient over the MDP state space, instead of estimating such a gradient over the model state space or some augmented space thereof. This is beneficial when considering MDP state spaces which are either strict subsets of the model state space or are smaller in dimension.

- Presented the result showing that pure proportional control on tracking error is prone to marginal stability. Thus, the decision was made to proceed with an MDP state space that spans 3 longitudinal states in addition to the tracking error: α, θ, q, e .
- Decided to attempt attitude tracking control and to focus on longitudinal control. So, the final control problem is pitch attitude tracking flight for the PH-LAB.
- Mentioned how detailed model comparison between Python and Matlab CitAST model should be presented in thesis.

Description of the results since last meeting

- **Runtime of the final algorithm** is 2.5 minutes per 90 seconds flight, profiling tests show that forward pass of neural networks and calculating weight updates take up most time (about 90 % of the time). Sadly cannot optimize those computations further. While individual runs take a long time, parallel processing would mean that results gathering could be sped up, but here there's another hurdle, to be mentioned in another bullet point.
- **Reconfigured code for pitch attitude control** and optimized the algorithm/flight setup for this new control problem:
 1. Changed MDP state space to span $\alpha, \theta, q, \theta_e$, by rewriting the output of the environment and the input layer of the IDHP networks.
 2. Did manual hyperparameter tuning of IDHP to gain grasp on the behaviour of the algorithm for such a tracking task. Found approximate values of hyperparameters which gave good enough performance.
 3. Experimented with varying the reference signal to be tracked, found that giving the agent an initial warmup phase and then performing "real flight manoeuvres" thereafter provided the best performance. Such a flight setup is a good compromise between having a realistic enough manoeuvre while providing the agent with the time for adjusting to controlling the aircraft.
- **Designed the final reference signal.** The flight duration lasts 90 seconds, the reference signal commands the aircraft to follow a sinusoidal signal for the first 40 seconds, and a pitch up pitch down manoeuvre for the last 30 seconds, see attached figure for actual time trace.

$$\theta_{ref}(t) = \theta_{trim} + \begin{cases} (10\sin(\frac{2\pi t}{15}) + 8\sin(\frac{2\pi t}{30}))(1 - \frac{t}{75}) & t < 45 \\ 1.5(t - 55) & 55 \leq t < 65 \\ 15 & 65 \leq t < 75 \\ -1.5(t - 85) & 75 \leq t < 85 \\ 0 & \text{otherwise} \end{cases} \quad [\text{deg}] \quad (1)$$

The reference pitch signal can be seen graphed in Figure 1.

- **Implemented the fault cases** to be used in the upcoming simulations: damaged elevator, shifted CG, and saturated elevator with damaged elevator.

The damaged elevator is modelled by scaling the magnitude of commanded elevator deflection by a factor of 0.3, meaning that the effective elevator control authority is capped at 30% of the nominal control authority.

Shifted CG is modelled by shifting the location of the CG, changing how the control and stability coefficients of the model are calculated.

The saturated and damaged elevator is modelled by first clipping the commanded elevator deflection to $[-5, 5]$ degrees, and then multiplying the clipped deflections by 0.3 before feeding the elevator command to the CitAST model. Thus, this results in the maximum and

minimum deflection that can be commanded by the agent to ± 5 degrees, and this saturated action is then scaled by 0.3 resulting in an effectively diminished elevator control authority.

- **Implemented Eligibility trace updates for the actor, and multi-step temporal difference errors for the critic.** Then performed Monte Carlo hyperparameter tuning by randomly sampling hyperparameters to see how various algorithms performed. Three metrics are gathered to quantify performance: Root Squared Error (RSE) in tracking during warmup phase, RSE during manoeuvre or in-flight phase, and power spectral density error between the reference signal and tracked state.

Unlike in research phase 1, where 15 runs were conducted for each hyperparameter, here only 1 run was conducted. This is because it was not possible to use multi-processing with the compiled python CitAST model, the python-multiprocessing library simply could not work with such files. So only 1 run was conducted for each hyperparameter. Thus runtime would scale linearly with the number of runs wanted. The final results are shown in ??, note that the x and y axes do not start at 0. Results show that there is a slightly higher probability for better tracking performance in IDHPAT and MIDHPAT when compared to IDHP.

With these results, it would have been logical to only keep IDHP, IDHPAT, and MIDHPAT as candidate algorithms to evaluate. However, since MIDHPAT is composed of MIDHP, it was deemed relevant to also analyse and present the performance of MIDHP. This does mean a slightly larger results section in the final thesis, which is a cost I've thus deemed to be adequate to bear. (thoughts?)

Subsequently, it was decided to keep IDHP, IDHP(λ) with accumulating traces, MIDHP, and MIDHP(λ) with accumulating traces for the final work.

The faults are to be triggered on the 60 second mark, as shown in Figure 1.

- **Performed a first stage of hyperparameter tuning** on the four selected algorithms, using the shifted CG fault flight case.

Found that all algorithms were likely to produce oscillatory control action, thus decided to implement Conditioning for Action Policy Smoothness (CAPS). After some fine-tuning, a compromise was reached between smooth enough actions and a good enough convergence rate (if CAPS was weighted too heavily, then agents found it difficult to follow the reference signal, but too little weight meant noisy actions).

- **Performed a second stage of hyperparameter tuning with CAPS incorporated.** Selected the best found hyperparameters, and then carried out Monte Carlo simulations of all four algorithms for four fault cases using their best hyperparameters. Preliminary plots of the four algorithms performance on the damaged elevator fault case are shown in Figure 2

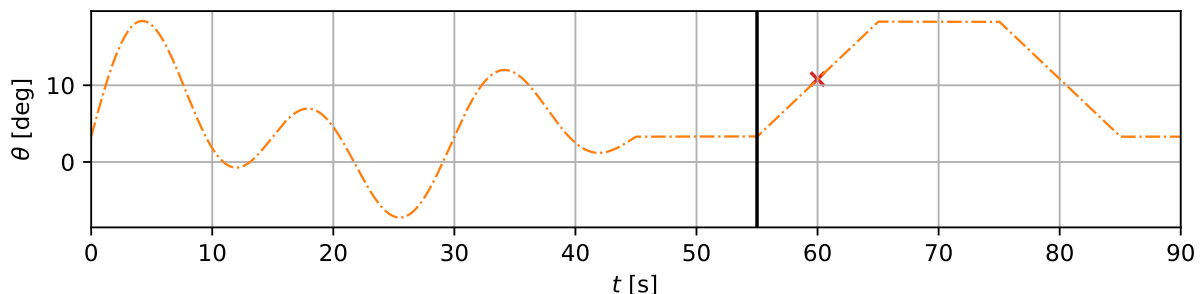


Figure 1: Reference pitch signal, the period before the black line will be referred to as the warmup phase, and afterwards is the inflight phase, the red cross at 60 s marks where faults will be triggered.

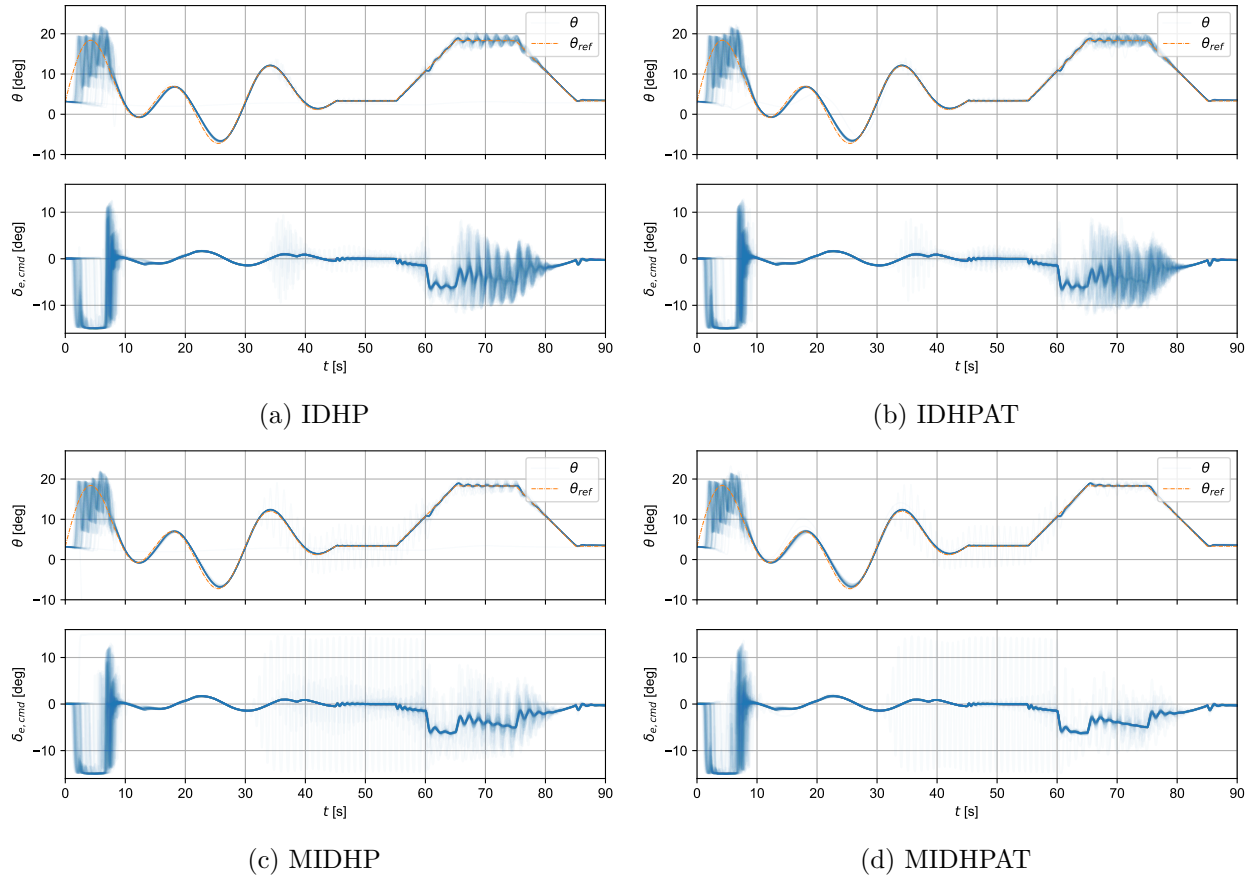


Figure 2: Preliminary plots showing performance of the four agents on the damaged elevator fault.

Plans for next 2 weeks

1. Work on data processing: gathering the warmup and in flight metrics. Namely, RSE and smoothness of state/action in two phases, also consider finding a nRSE, or perhaps moving to normalised mean absolute errors (nMAE)?

Figure out a good way to plot the time traces of each agent. Using the *μanalysis* approach and just plot opaque trajectories of each flight? Plot the mean state value with an upper and lower standard deviation bound? Plot mean with minimum and maximum bounds?

2. Begin drafting scientific article.